

## **Deliverable: Testing**

Cohort 1, Team 8

### **Team Members:**

Michael Coleman

Finn Done

Charlie Webster

Philip McGowan

Hammad Sami

Tolly Shearman

Eric Vokes

## Methods and Approaches

In order to test our game we opted for a hybrid approach, this being through both manual and automated testing. Where possible we opted for our tests to be automated, this ensured that these select tests could be re-ran after any new changes were made during the implementation process. Additionally this process sped up the implementation time by increasing developer confidence with new commits where our automated tests acted as a safety net to ensure stability.

One factor we took into consideration while testing was coverage. In order to do this we ensured that every likely and important scenario that could occur is tested. While we looked to have as high a coverage as possible, it was necessary that we did not attempt to complete 100% coverage as this would have been impractical and in some cases could even be impossible. Due to the complexity of the game, many different scenarios could occur that may not have been thought of with initial testing. This is why user evaluation is important, so that users can have a chance to play the game and interact with it in ways that we may not have intended.

Because of how our game was implemented using LibGDX, it was very impractical to attempt to have a high coverage using automated testing alone. A reason for this difficulty is that we decided to implement a lite version of a game engine, where as well as allowing editing the map through Tiled, objects could also be placed with their variables defined in real time. This resulted in the necessity to call for these variables within the constructor for each class which was not compatible with unit testing. We believed that this sped up development time more efficiently than the extensive use of automated testing could. One possible way to get around problems caused by LibGDX was via the use of Mockito, which allowed us to write automated tests for both the hotbar and the leaderboard. However, with other objects in the game we saw that the complexity for mocking was much larger and not possible to implement with the current scope of the project. Furthermore the use of mocks was avoided since they couldn't effectively simulate real functionality and interaction that is experienced within the game environment. As a result, despite our preference for automated testing, if the mocks were becoming too complex we would move towards a manual approach for the class instead.

Ensuring we tested functionalities at different levels was vital for us to know that all parts of the system were working as intended. We can organise these tests into levels depending on their scope. The first of these being unit tests, which tested individual functionality such as a class or method. Secondly we also used integration testing. This scope tested the interaction between adjacent components in the game, making sure that classes or methods that worked in isolation were also working together correctly. Lastly, end-to-end tests are done to make sure that the whole system works together, building on top of the tests of previous scopes to test their interactions overall.

**Comment on completeness and correctness:** On the final games files all of the automated and manual tests passed. Some had previously failed due to previous changes but the updates were made to fix the bugs in the actual game code. The manual tests are complete as they test all front facing classes and vice versa with manual, which we can confirm as being forrest due to the headless mocking which isolates it from the system.

## Manual Tests

| Description                                      | Component / Requirement                          | Category               | Steps                                                          | Expected outcome                                                                | Status |
|--------------------------------------------------|--------------------------------------------------|------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------|--------|
| Check map loads correctly                        | MapLoader / FR_MAP                               | End to End - MapLoader | Launch game.                                                   | Map loads with correct layers, e.g. all objects should be rendered as expected. | Pass   |
| Check that movement works as intended.           | Player / UR_IMMERSION                            | Unit - Player          | Walk in each direction (up down left right) and the diagonals. | Player should moved in direction pressed with consistent speed                  | Pass   |
| Check that player speed increases with modifiers | Player, PlayerSpeed Trigger / FR_POSITIVE_EVENTS | Integration            | Interact with a speed boost event.                             | The player's speed increases temporarily and is still consistent.               | Pass   |
| Check Player collisions                          | Player / UR_IMERSON                              | Integration            | Walk into the world border and collidable at every angle.      | Players speed should be set to 0.                                               | Pass   |
| Footstep audio                                   | Player / FR_SOUND                                | Unit                   | Walk for a few seconds and by one key tap.                     | Audio should be played only while moving.                                       | Pass   |
| Check the enemy path following.                  | Enemy / FR_NEGATIVE_EVENTS                       | Integration            | Set path for enemy and follow behind in game.                  | The enemy should follow the path defined.                                       | Pass   |
| Check enemy player tracking                      | Enemy / FR_NEGATIVE_EVENTS                       | Integration            | Walk within range of the enemy and then run away.              | Enemy should track and follow player.                                           | Pass   |
| Check enemy collisions.                          | Enemy / FR_NEGATIVE_EVENTS                       | Integration            | Walk within range and run round a corner.                      | Enemy should collide with wall instead of passing through.                      | Pass   |
| Test intractable                                 | Interactable.                                    | Integration.           | Create an interactable and press E, e.g. Long boi.             | Entity should trigger an outcome, in this case the dialogue.                    | Pass   |
| Check functionality of giving                    | GiveTakeItemTrigger / FR_EVENTS                  | Integration            | Interact with a chest.                                         | Key should now be visible in the next available inventory                       | Pass   |

| items.                                   | _TYPE                                    |              |                                                                                | slot                                                                                                                              |      |
|------------------------------------------|------------------------------------------|--------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------|
| Check functionality of removing items.   | GiveTakeItemTrigger                      | Integration  | Interact with a door.                                                          | Key should now be removed from the inventory                                                                                      | Pass |
| Check that dialogue triggers.            | DialogueTrigger, Dialogue / UR_IMERSION  | Integration  | Interact with the enemy entity that starts dialogue.                           | The dialogue box should open when interacting.                                                                                    | Pass |
| Check dialogue typewriter effect         | Dialogue / UR_IMERSION                   | Unit         | Interact to open a dialogue box.                                               | Letters should come in one by one until full text is displayed.                                                                   | Pass |
| Check sound work in dialogue box         | Dialogue / FR_SOUND                      | Unit         | Interact to open a dialogue box.                                               | Sounds should play while the typewriter effect occurs. .                                                                          | Pass |
| Check dialogue text.                     | Dialogue / UR_IMERSION                   | Integration  | Interact with professor.                                                       | Dialogue displayed should match that of the relevant file, "professor.txt" in this case.                                          | Pass |
| Test closing dialogue                    | Dialogue / UR_IMERSION                   | Unit         | Attempt closing twice independently with space and enter key.                  | The dialogue box should close on each key respectively.                                                                           | Pass |
| Test pause menu opening / closing        | PauseMenu / FR_PAUSE_MENU                | Integration  | Launch game and press p, esc, and left click respectively to pause and unpaue. | The game should start paused and pressing each button should result in both pausing and unpausing.                                | Pass |
| Check pause menu text                    | PauseMenu / FR_PAUSE_MENU                | Integration  | Launch game and pause menu.                                                    | Text displayed should match that of "PAUSED" + the contents of the file "tutorial.txt" and should be in the centre of the screen. | Pass |
| Check leaderboard displayed when paused. | PauseMenu / Leaderboard / FR_LEADERBOARD | Integration. | Launch game with specific values entered into "leaderboard.txt"                | The leaderboard should be displayed while paused to the left of the menu and text should match that                               | Pass |

|                                           |                                                             |             |                                                                |                                                                                                                    |      |
|-------------------------------------------|-------------------------------------------------------------|-------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|------|
|                                           |                                                             |             |                                                                | in file.                                                                                                           |      |
| Test Levers                               | LeverPullTrigger /<br>FR_NEGATIVE_EVENTS                    | Integration | Pull lever that is set to lower spikes                         | Lever should be set to pulled sprite and spikes should go down.                                                    | Pass |
| Test winning the game.                    | GameWinTrigger /<br>FR_WINNABLE,<br>FR_END_SCREEN           | End to end  | Run through full game to finally give sausage roll to long boi | Game over screen should be displayed containing the "win" test, a summary of the time, score, and events completed | Pass |
| Test losing the game                      | FR_END_SCREEN                                               | End to end  | Launch the game, unpause and wait 5 minutes.                   | Game over screen should be displayed saying "loss" instead.                                                        | Pass |
| Test camera systems                       | CameraSystems /<br>UR_IMMERSION                             | Integration | Move around in the game map .                                  | Camera should follow player around.                                                                                | Pass |
| Check status bar score and event counter. | StatusBar /<br>UR_SCORE,<br>UR_FEEDBACK                     | Unit        | Interact with a star which gives 250 points.                   | Status bar should update score to +250 and events to +1                                                            | Pass |
| Check status bar time count.              | StatusBar /<br>FR_TIMER,<br>FR_TIME_LIMIT                   | Unit        | Launch game, unpause and start external timer.                 | After 5 minutes has passed on the external timer, the same should be shown in game.                                | Pass |
| Test toast bar.                           | ToastBar/<br>FR_ACHIEVEMENT_LIST,<br>FR_ACHIEVEMENT_TRIGGER | Integration | Interact with one, and then multiple items to initiate toast.  | Toast should pop up and then fade, if multiple at the same time they stack.                                        | Pass |

## Automated Tests

For our automated tests we focused on testing components that contain high amounts of pure logic, rather than a lot of rendering, these being the leaderboard and the hotbar which link to FR\_LEADERBOARD and UR\_IMMERSION respectively. It was important to test the hotbar automatically especially as a lot of other components rely on the fact that the player has an inventory and can pick up, use, and know what items they currently have.